
AI-driven software engineering

Architecting Software Systems

AI-Driven Software Engineering

Spec-driven engineering, agentic programming, token optimization and human quality control

- From prompt-based coding to controlled software evolution
- Requirements · specs · agents · context · tests · review · observability
- A course about using AI without losing engineering discipline

AI-SE

specs
agents
context
tokens
tests
review

2026

Why This Subject?

AI changes the cost of producing code, not the need for engineering

- Code generation is becoming cheaper and faster
- Understanding, validating and evolving code remain difficult
- The bottleneck moves from typing code to specifying intent and checking results
- Students must learn to supervise AI systems, not only use them
- Good software engineering becomes more important when code is generated quickly

AI does not remove software engineering. It raises the standard for it.

Course Map

Four connected blocks

Spec-driven engineering

Write requirements, designs, tasks and acceptance criteria before implementation

Agentic programming

Use AI agents that inspect, edit, test and iterate over codebases

Token and context optimization

Control what the model sees, how much it sees and why

Human quality control

Testing, review, security, traceability and responsible use

Part I

From prompting to spec-driven engineering

- A prompt is often too small to capture engineering intent.
- Specifications create a shared contract between humans, AI agents and tests.

The Problem with Prompt-Only Development

Vague intent produces fragile software

```
Bad prompt:  
"Add CSV export."
```

```
Missing information:  
- Which command or screen exposes it?  
- What is the expected file format?  
- What happens with missing fields?  
- Which existing behaviour must not change?  
- How will we test it?
```

- The AI fills gaps with assumptions
- Hidden assumptions become bugs
- Review becomes difficult because intent was never written
- The same prompt can produce different designs

Spec-Driven Engineering

Treat intent as a first-class artifact

- A feature starts with a written specification
- The spec explains the problem, expected behaviour and non-goals
- Tasks decompose the change into reviewable steps
- Tests validate the specification
- Implementation becomes one artifact among several

In AI-driven development, the spec is the source of control.

Spec Artifacts

A lightweight structure for AI-assisted changes

```
change-add-csv-export/  
  proposal.md  
  design.md  
  tasks.md  
  specs/  
    report-export.md  
  tests/  
    acceptance-cases.md
```

- proposal.md: why the change is needed
- design.md: architecture and trade-offs
- tasks.md: implementation checklist
- specs/: observable behaviour and acceptance criteria

Open-Source SDD Projects

Spec-driven engineering is becoming a practical tool ecosystem

OpenSpec	Lightweight spec layer for agreeing what to build before code; organizes changes with proposal, specs, design and tasks
GitHub Spec Kit	Open-source toolkit for spec-driven development with AI coding agents and structured flows from intent to implementation
Common idea	Move ambiguity out of chat and into reviewable artifacts
Teaching value	Students can compare different workflows instead of treating SDD as only a theory

OpenSpec vs Spec Kit

Two different styles of spec-driven development

OpenSpec style

Lightweight, iterative, useful for existing/brownfield projects and change folders

Spec Kit style

More guided flow with specification, clarification, planning, tasks and implementation

Both

Help human and AI align before code is generated

Course exercise

Implement the same small feature using both styles and compare effort, clarity and review quality

Example: proposal.md

Motivation, scope and non-goals

```
# Add CSV report export

## Motivation
Teachers need to upload reports to external systems.

## Proposed change
Add a CSV exporter and expose it through --format csv.

## Non-goals
Do not modify PDF export.
Do not change the internal Report model.
Do not add a web interface.
```

- A proposal limits the scope
- Non-goals prevent accidental over-engineering
- The AI agent can be instructed to stay inside the proposal

Example: requirements as acceptance criteria

Make behaviour testable

Requirement: export reports as CSV

Acceptance criteria:

- Given a report with three students
- When the user runs `--format csv`
- Then the output contains one header row
- And one row per student
- And missing grades are empty cells
- And PDF export still works

- Acceptance criteria connect requirements to tests
- They define observable behaviour
- They help humans review AI-generated code

Example: tasks.md

Small steps for human and agent execution

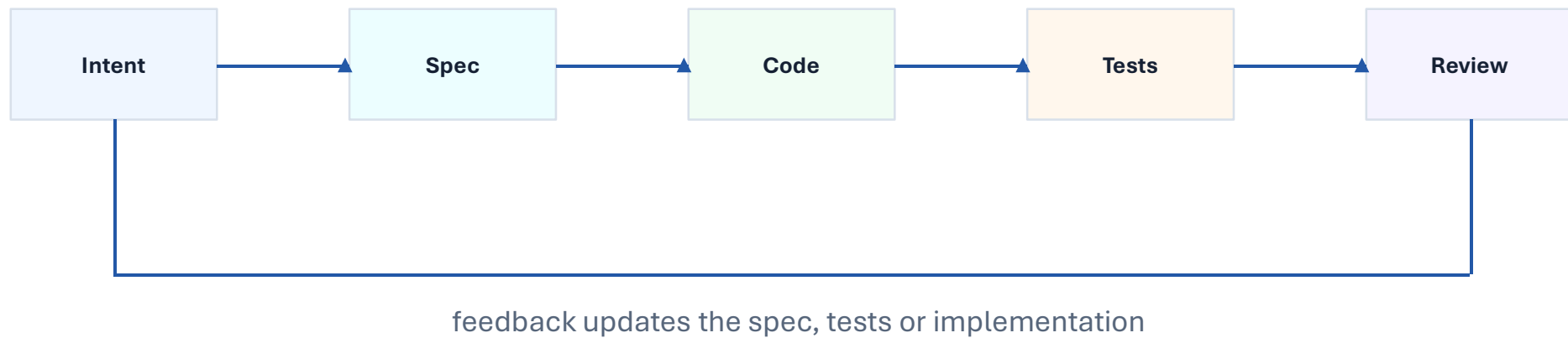
Tasks

- [] Create CsvReportExporter
- [] Make it implement ReportExporter
- [] Add --format csv to the CLI parser
- [] Add tests for header and row generation
- [] Add test for missing grades
- [] Run the existing PDF tests
- [] Update README usage examples

- Tasks reduce the change into small units
- Each task can be reviewed independently
- Agents perform better with concrete, bounded tasks

Spec-to-Code Loop

A controlled AI-assisted workflow



- Start from intent
- Write a specification
- Generate or edit code
- Run tests and checks
- Review and update the spec if needed

Spec-Driven vs Test-Driven

Complementary, not competing

Spec-driven	Clarifies intent, scope, constraints and acceptance criteria
Test-driven	Turns expected behaviour into executable feedback
Together	The spec explains what should happen; the tests prove it happens
With AI	The agent can generate code, but tests and specs constrain the result

Part II

Agentic programming

- An AI assistant answers questions.
- An AI agent plans, uses tools, edits files, runs tests and iterates.

From Assistant to Agent

The difference is tool use and iteration

Assistant

Explains, suggests, drafts code snippets and answers questions

Copilot-style assistant

Completes code inside the IDE using local context

Coding agent

Reads files, edits code, runs tests, debugs and repeats

Autonomous workflow

Plans tasks, uses tools and reports results under human supervision

Agentic Programming Loop

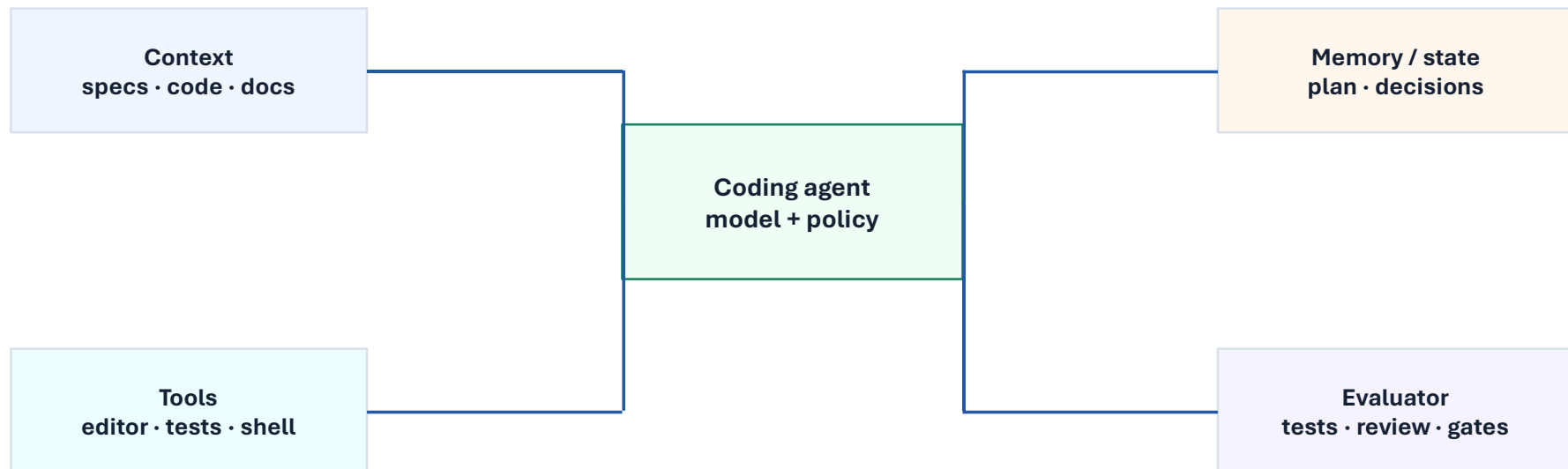
Plan, act, observe, repair

```
while (!taskDone) {  
    planNextStep();  
    editFiles();  
    runTests();  
    inspectFailures();  
    repairOrAskHuman();  
}  
  
// The loop must be bounded by specs, tests and review.
```

- The agent works through iterations
- Tools provide feedback: compiler, tests, linters, logs
- The human defines goals, constraints and stop conditions

Agent Architecture

A simple mental model



- Model: reasoning and generation
- Context: files, specs, docs and conversation
- Tools: shell, tests, search, editor, browser
- Memory/state: what the agent knows across steps
- Evaluator: tests, checks and human review

Example Agent Task

A bounded task is easier to automate safely

```
Task:  
Implement CsvReportExporter following specs/report-export.md.
```

```
Constraints:
```

- Do not modify PdfReportExporter
- Keep public Report API unchanged
- Add tests before implementation
- Run all tests before finishing
- Summarize changed files and risks

- The task tells the agent where to work
- Constraints prevent unnecessary refactoring
- Required verification makes completion observable

Good Agent Tasks

Small, testable and reviewable

- One clear outcome
- Explicit files or modules when possible
- Acceptance criteria and examples
- Constraints and non-goals
- Required commands: tests, formatters, static checks
- Expected final report: changes, tests, limitations

Bad Agent Tasks

Too broad, vague or unverifiable

```
Bad:
"Improve the app."
"Make the code better."
"Refactor everything."
"Fix all bugs."

Better:
"Extract CSV parsing from ReportService into CsvImporter.
Keep behaviour unchanged and run ReportServiceTest."
```

- Broad tasks invite unnecessary changes
- Vague goals are hard to verify
- Unbounded refactoring increases review cost

Human-in-the-Loop Agentic Development

Decide what the agent may do alone

Low risk	Generate tests, explain code, update docs, small local refactorings
Medium risk	Implement a bounded feature with tests and review
High risk	Security changes, data migrations, authentication, production deploys
Human control	Approval gates, code review, test evidence and rollback plans

Part III

Token and context optimization

- AI tools are limited by context, attention and cost.
- Advanced users learn to provide the right information, not all information.

Why Tokens Matter

Tokens are the budget of AI-assisted development

- Every prompt, source file, log and generated response consumes tokens
- Large context can increase cost and latency
- Irrelevant context can distract the model
- Missing context leads to hallucinated assumptions
- Token optimization is about selecting useful evidence

More context is not always better. Better context is better.

Context Engineering

The skill of building the right prompt context

Goal	What change or answer is needed?
Relevant code	Files, interfaces, tests and examples that constrain the task
Project rules	Style, architecture, commands, quality gates
Evidence	Errors, logs, failing tests, user stories, acceptance criteria
Exclusions	Files and behaviours that should not change

Token Optimization Techniques

Practical ways to reduce noise

- Use summaries for stable background information
- Include exact code only for files that matter
- Provide failing test output instead of full logs
- Use file maps before sending complete files
- Prefer diffs for recent changes
- Ask the agent to inspect before editing

Example: Bad Context

Too much information, too little structure

```
Prompt:  
Here is the whole repository. Fix CSV export.
```

```
Problems:  
- The model sees many irrelevant files  
- Important requirements are not highlighted  
- It may edit unrelated modules  
- Review becomes harder
```

- Large context can hide the important parts
- The agent may optimize locally but damage architecture
- Unclear context leads to unclear changes

Example: Better Context

Small, structured and testable

Goal: add CSV export.

Relevant files:

- ReportExporter.java
- PdfReportExporter.java
- ReportCli.java
- ReportExporterTest.java

Spec:

- --format csv creates header + one row per student
- missing grades are empty cells
- PDF behaviour must remain unchanged

Verification:

- run mvn test

- The model receives the real constraints
- The scope is explicit
- Verification is part of the prompt

Retrieval for Codebases

Finding the right context automatically

- Keyword search: find names, errors and APIs
- Symbol search: find definitions and references
- Embeddings: find semantically related code or documentation
- Dependency graph: follow imports, callers and tests
- Hybrid retrieval: combine lexical, semantic and structural signals

CodeRAG is useful only if retrieved context is precise and trustworthy.

Token Optimization for Agents

Agents should not reread the whole world every step

Memory

Keep concise summaries of decisions and constraints

Scratchpad

Track current plan, changed files and open questions

Retrieval

Fetch only relevant files or snippets when needed

Compression

Summarize logs, diffs and long documents

Checkpoints

Stop after meaningful milestones for human review

Part IV

Human control and quality

- AI-generated code must be treated as untrusted until validated.
- Quality is created through tests, review, traceability and observability.

The Human Role

From coder to reviewer, architect and product thinker

- Define the problem and expected value
- Write or approve the specification
- Select the architecture and constraints
- Review generated code and tests
- Judge trade-offs, maintainability and risk
- Take responsibility for the final system

Quality Gates for AI-Generated Code

Make acceptance explicit

A change is not accepted until:

```
[ ] It satisfies the written spec
[ ] All existing tests pass
[ ] New behaviour has tests
[ ] Static analysis has no critical issue
[ ] Security-sensitive code was reviewed
[ ] The diff is small enough to understand
[ ] The agent explains residual risks
```

- Quality gates reduce trust-by-default
- They make review repeatable
- They teach students professional discipline

Testing AI-Generated Software

Tests are the main feedback channel

Unit tests

Check isolated behaviours and edge cases

Integration tests

Check collaboration between components

Regression tests

Protect existing behaviour from AI changes

Mutation tests

Evaluate whether tests detect small code changes

Acceptance tests

Validate user-facing requirements from the spec

Review Checklist

What to inspect in AI-generated code

- Does the change actually implement the spec?
- Did it modify unrelated files?
- Are names and abstractions consistent with the codebase?
- Are edge cases and errors handled?
- Are tests meaningful or only increasing coverage?
- Does the design increase coupling or hidden state?
- Are security and privacy assumptions explicit?

Security and Privacy Risks

AI may generate plausible but unsafe code

- Hardcoded secrets or credentials
- Missing input validation
- SQL injection or unsafe command execution
- Overly broad permissions
- Leaking personal data in logs or prompts
- Copying sensitive code into external tools without policy approval

Security-sensitive changes require stronger human review.

Traceability

Connect intent, code and evidence

```
User story -> spec -> tasks -> commits -> tests -> review
```

Example:

US-014 Add CSV export

spec: specs/report-export.md

tasks: tasks.md items 1-6

tests: CsvReportExporterTest

PR: #42

evidence: mvn test, coverage report

- Traceability explains why code exists
- It supports audits and maintenance
- It helps compare AI output against human intent

Observability for AI-Assisted Development

Measure the process, not only the code

Prompt/spec quality	How often did the agent need clarification?
Task success	Did tests pass without human repair?
Review findings	How many defects were found in generated code?
Change size	Was the diff small and understandable?
Regression rate	Did the change break unrelated behaviour?

Part V

Course labs and assessment ideas

- Students should practice supervised AI development, not passive code generation.
- The deliverables should include specs, code, tests and reflection.

Lab 1: Spec Before Code

Turn a vague request into an engineering artifact

Vague request:

"Add notifications when students fail."

Deliverable:

A small spec that another student or AI agent can implement.

- Start from a vague feature request
- Write proposal.md, design.md and tasks.md
- Define acceptance criteria and non-goals
- Ask an AI assistant to critique the spec
- Revise the spec before implementation

Lab 2: Agentic Implementation

Use an agent under constraints

```
Agent instruction:  
Implement task 2 only.  
Do not modify public APIs.  
Run mvn test.  
Report changed files and remaining risks.
```

- Give the agent one bounded task from tasks.md
- Require tests before or with implementation
- Run test and lint commands
- Review the diff manually
- Reject or revise changes that do not match the spec

Lab 3: Token Optimization

Same task, different contexts

- Prepare a large noisy context and a small curated context
- Ask the AI to solve the same task with both
- Compare output quality, errors and review effort
- Measure prompt size and generated diff size
- Discuss why context selection matters

Lab 4: Human Quality Review

Audit AI-generated code

- Receive an AI-generated pull request
- Check it against the spec
- Run tests and inspect coverage
- Look for security, coupling and maintainability issues
- Write a review with required changes
- Decide whether to accept, revise or reject

Possible Assessment

Evaluate the whole AI-driven engineering process

Specification	Clarity, acceptance criteria, constraints, non-goals
Implementation	Correctness, design quality, integration with existing code
Verification	Tests, CI evidence, static analysis, edge cases
Review	Quality of human critique and risk assessment
Reflection	What AI helped with, what failed, what the student changed

Final Project Idea

A controlled AI-assisted feature extension

Final deliverables:

- proposal.md
- design.md
- tasks.md
- tests and implementation
- review report
- AI usage and quality reflection

- Choose an existing small codebase
- Define one meaningful feature through a spec
- Use an AI agent to implement bounded tasks
- Use tests, quality gates and review to validate
- Deliver code plus a process report
- Explain how token/context choices affected the result

Key Takeaways

AI-driven software engineering is still engineering

- Specifications make AI-assisted development controllable
- Agents need bounded tasks, tools, tests and stop conditions
- Token optimization is really context engineering
- Human review remains essential for quality, security and architecture
- The best students will learn to combine AI speed with engineering judgment

The goal is not to generate more code. The goal is to evolve better software with control.